

Звіт про виконання лабораторної роботи №4

Розробка вебсерверів засобами фреймворку ASP.NET.

Виконав: студент групи 201-пТК
Ткаченко Микола Віталійович

Запитання	Відповідь
ASP.NET Core Web API vs MVC/ Blazor	Web API повертає лише дані (зазвичай у форматі JSON/XML) і призначений для споживання іншими клієнтами (фронтенд, мобільні додатки). MVC повертає готові HTML-сторінки (View). Blazor використовує WebAssembly або SignalR для створення інтерактивних клієнтських UI на C#.
Трирівневаархітектура	Складається з: 1. Presentation Layer (API Контролери). 2. Business Logic Layer (Сервіси). 3. Data Access Layer (Репозиторії/EF Core). Це забезпечує гнучкість, легкість тестування та розділення відповідальності.
Моделі (DTO) vs Сутності БД	Моделі у REST (DTO - Data Transfer Objects) потрібні для передачі даних клієнту. Вони приховують внутрішню структуру БД, паролі, циклічні залежності та зайві поля, які клієнту бачити не потрібно.

REST — це архітектурний стиль. Однорідність інтерфейсу означає використання стандартних HTTP методів, унікальних

1. Опис реалізації

Згідно з завданням створено проєкт **SmartHome.REST** (ASP.NET Core Web API).

- **Папка Models:** Створено Data Transfer Objects (DTOs), зокрема `LightBulbDTO`, які відокремлені від моделей бази даних.
- **Controllers:** Створено `DevicesController` із реалізацією повного CRUD (GET, POST, PUT, DELETE).
- **Пагінація:** Реалізовано через Query-параметри `page` та `amount` у методі `GetAll`.
- **Swagger:** Підключено для тестування API інтерфейсу.

2. Відповіді на контрольні запитання

4. REST і Uniform Interface	URI для кожного ресурсу (напр. `/api/devices/1`) та використання стандартних форматів (JSON).
5. HTTP-методи для CRUD	Create -> POST (`[HttpPost]`). Read -> GET (`[HttpGet]`). Update -> PUT або PATCH (`[HttpPut]`). Delete -> DELETE (`[HttpDelete]`).
6. HTTP-коди відповідей	200 OK (успіх GET/PUT), 201 Created (успіх POST), 204 No Content (успіх DELETE/PUT), 400 Bad Request (помилка валідації), 404 Not Found (ресурс не знайдено), 500 Internal Server Error (помилка сервера).
7. Асинхронні методи ICrudServiceAsync	Вони повертають `Task`. Під час операцій вводу/виводу (звернення до БД) потік не блокується, а повертається в пул потоків. Це критично підвищує пропускну здатність сервера при великих навантаженнях.
8. Dependency Injection (DI)	Це патерн, де фреймворк автоматично створює та передає залежності (напр., сервіси у контролери). Реєстрація відбувається в `Program.cs` через `builder.Services.AddScoped()` тощо.
9. Пагінація	Розбиття великого масиву даних на частини (сторінки) за допомогою `.Skip().Take()`. Потрібна для зменшення навантаження на мережу, сервер та БД, оскільки клієнту не відправляються відразу тисячі записів.

10. Тестування API

Тестування проводилося за допомогою вбудованого інструменту **Swagger UI**, який автоматично генерує сторінку з документацією та можливістю відправляти HTTP-запити.
Також можна використовувати Postman.